



مهلت : ۴ آذر

شبکه های کامپیوتری

Fall 2025



تمرین سوم

۱ نقش شماره پورت

لایه شبکه (IP) مسئول تحویل Host-to-Host (میزبان به میزبان) است. توضیح دهید که چگونه لایه انتقال، با استفاده از مفهوم شماره پورت، این سرویس را گسترش می دهد تا به تحویل Process-to-Process (فرآیند به فرآیند) دست یابد.

۲ نقش کنترل جریان در شبکه

TCP از دو مکانیسم کنترلی متمایز، کنترل جریان (Control Flow) و کنترل ازدحام (Control Congestion) استفاده می کند که هر دو نرخ ارسال فرستنده را محدود می کنند.

۱. هدف اصلی کنترل جریان را بیان کنید و توضیح دهید که از کدام جزء در شبکه (فرستنده، گیرنده یا روترها) محافظت می کند.

۲. هدف اصلی کنترل ازدحام را بیان کنید و توضیح دهید که از کدام جزء در شبکه محافظت می کند.

۳ UDP

پروتکل دیتگرام کاربر (UDP) اغلب به عنوان یک پروتکل انتقال "بی آرایش" یا بسیار ساده توصیف می شود زیرا فاقد خدمات اصلی مانند تحویل تضمین شده و کنترل ازدحام است. با این حال، همچنان دو سرویس اساسی لایه انتقال را ارائه می دهد که برای هر ارتباطی ضروری هستند.

این دو سرویس اساسی که UDP ارائه می دهد را شناسایی کرده و به طور خلاصه توضیح دهید.

۴ جمع‌آوری و تفکیک (Multiplexing/Demultiplexing) در UDP

UDP یک پروتکل بدون اتصال (Connectionless) و بدون سربرار است. با این حال، باید داده‌های لایه شبکه (IP) را به فرآیند کاربردی صحیح در لایه بالا هدایت کند.

۱. فرستنده (Sender) چگونه عمل جمع‌آوری (Multiplexing) را انجام می‌دهد تا بتواند چندین فرآیند کاربردی را در یک میزبان مدیریت کند؟

۲. گیرنده (Receiver) چگونه از عمل تفکیک (Demultiplexing) برای تحویل صحیح سگمنت‌های دریافتی به فرآیند کاربردی مورد نظر استفاده می‌کند؟

۳. مشخص کنید که UDP در هر سگمنت، به چند شناسه (شماره پورت) برای انجام عملیات تفکیک نیاز دارد؟

۵ توالی و شماره تأییدیه در TCP

TCP برای تضمین تحویل مطمئن و مرتب، از شماره توالی (Number Sequence) و شماره تأییدیه (Acknowledgment Number یا ACK) استفاده می‌کند

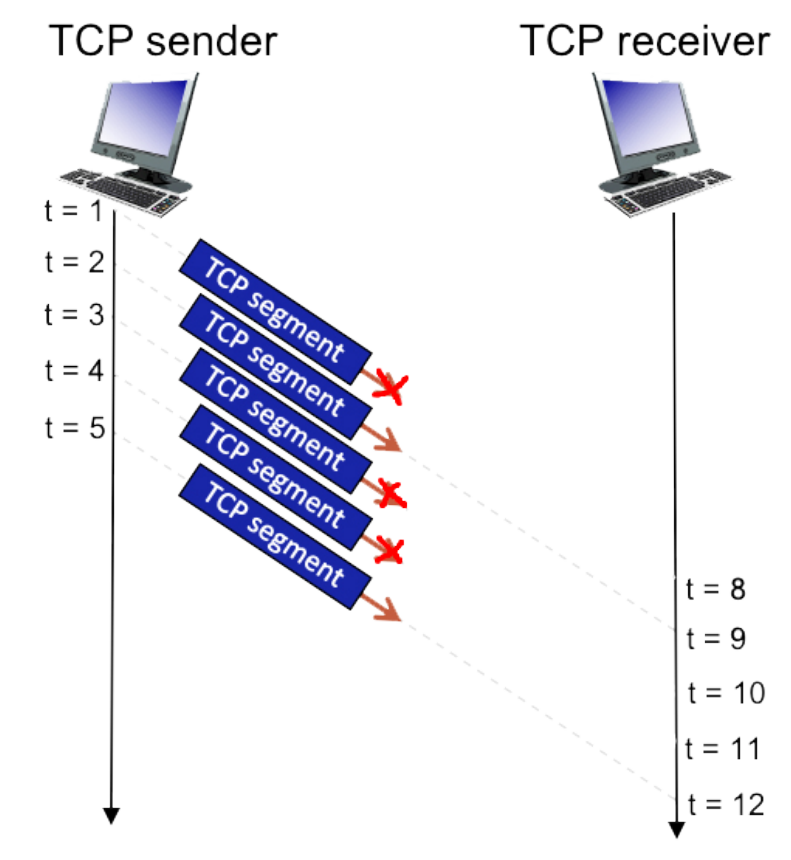
۱. در TCP، شماره توالی یک سگمنت به کدام جزء از داده (بایت) اشاره دارد و چگونه محاسبه می‌شود؟

۲. شماره تأییدیه (ACK) در یک سگمنت TCP، چه چیزی را به فرستنده اعلام می‌کند (پاسخ باید شامل مفهوم ACK تجمعی باشد)؟

۳. اگر فرستنده یک سگمنت با شماره توالی 1000 و اندازه 100 بایت را ارسال کند، شماره توالی سگمنت بعدی که فرستنده ارسال خواهد کرد، چند است؟

۶ Loss Segment

به شکل زیر توجه کنید که در آن یک فرستنده و گیرنده TCP بر روی یک اتصال با هم ارتباط برقرار می‌کنند که در آن بسته‌های sender->receiver ممکن است از دست بروند. فرستنده TCP یک initial پنجره به اندازه 5 segments ارسال می‌کند. فرض کنید مقدار اولیه شماره توالی sender->receiver برابر 402 باشد و هر یک از پنج بسته اول شامل 343 bytes باشد. تأخیر بین فرستنده و گیرنده 7 units time است، بنابراین اولین segment در زمان $t=8$ به گیرنده می‌رسد. همان‌طور که در شکل نشان داده شده است، 3 از 5 segment(s) بین فرستنده و گیرنده از دست رفته‌اند.



سؤال ۱: شماره‌های توالی مربوط به هر یک از 5 segments که توسط فرستنده ارسال شده‌اند را بدهید.

سؤال ۲: شماره‌های ACK که گیرنده در پاسخ به هر یک از segments ارسال می‌کند را بدهید.

۷ رسم FSM برای گیرنده در rdt 0.3

سؤال: FSM (machine state finite) را برای سمت receiver پروتکل rdt 0.3 رسم کنید.

۸ چالش لینک ماهواره‌ای با سرعت بالا (Transfer Data Reliable / Pipelining)

یک تیم تحقیقاتی در حال پیاده‌سازی یک protocol انتقال داده سفارشی بر روی یک لینک ماهواره‌ای جدید با high-speed و فاصله طولانی است. لینک دارای Bandwidth (R) برابر با 10 Gbps و Time Round-Trip (RTT) بسیار زیاد 600 ms است. مهندسين شبکه باید بین دو Go-Back-N protocol (GBN) و Selective Repeat (SR) برای اطمینان از reliability یکی را انتخاب کنند. سناریو: لینک دارای Rate Error Packet non-zero but low (p) برابر با 1×10^{-4} است (یعنی یک بسته از ده هزار بسته ممکن است از دست برود یا خراب شود). وظیفه شما به عنوان یکی از اعضای این تیم به شرح زیر است:

۱. محاسبه Product Bandwidth-Delay (BDP) این لینک و توضیح اهمیت آن در زمینه RDT protocols.
۲. با فرض یک پیاده‌سازی ایده‌آل Go-Back-N، bottleneck performance اصلی ناشی از BDP high و نرخ خطای غیر صفر loss packet را شرح دهید.
۳. توضیح دهید که چرا Repeat Selective (SR) برای این لینک خاص گزینه برتر است، حتی اگر پیاده‌سازی آن پیچیده‌تر از GBN باشد.

داده‌ها:

$$\diamond \text{ Bandwidth (R): } 10\text{Gbps} = 10 \times 10^9 \text{bits/second}$$

$$\diamond \text{ RTT: } 600\text{ms} = 0.6\text{seconds}$$

$$\diamond \text{ Segment size (L): } 1500\text{bytes} - \text{header including}$$

$$\diamond \text{ Rate Error Packet (p): } 1 \times 10^{-4}$$

راهنمایی:

- ◇ BDP نشان‌دهنده میزان کل داده‌ای است که می‌تواند همزمان در لینک باشد (پر کردن "pipe") قبل از اینکه اولین ACK دریافت شود.

- ◊ در GBN، اگر یک بسته از دست برود، تمامی بسته های بعدی که به درستی دریافت شده اند دوباره ارسال می شوند که در لینک های با BDP high و loss packet non-zero باعث کاهش شدید throughput می شود.
- ◊ SR امکان ارسال مجدد تنها بسته های از دست رفته را می دهد و بنابراین بهره وری لینک در چنین شرایطی بسیار بالاتر است.

۹ شماره های پورت در یک اتصال TCP

یک اتصال TCP بین A Host و B Host را در نظر بگیرید. فرض کنید TCP segments که از A Host به B Host ارسال می شوند دارای x number port source و y number port destination هستند.

سؤال: شماره های source و port destination برای segments که از B Host به A Host ارسال می شوند چیست؟

۱۰ RDT: پیاده سازی یک پروتکل انتقال قابل اعتماد

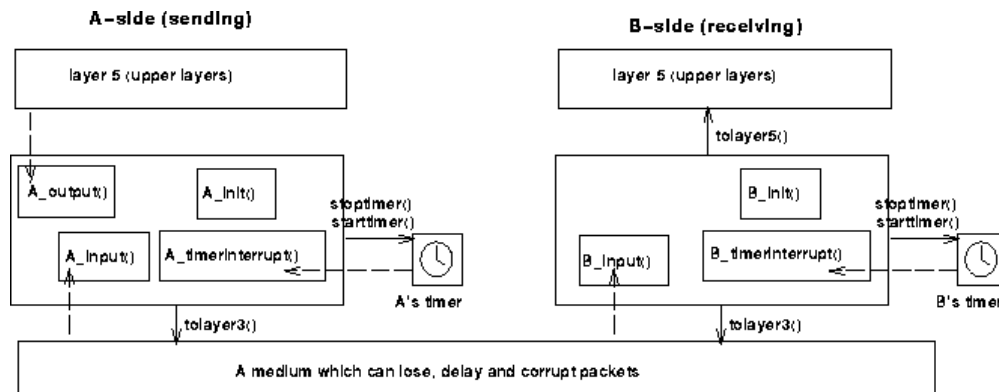
نمای کلی

در این تکلیف برنامه نویسی آزمایشگاهی، شما کد سمت فرستنده و گیرنده لایه انتقال را برای پیاده سازی یک پروتکل انتقال داده قابل اعتماد ساده خواهید نوشت. این آزمایشگاه باید جالب باشد زیرا پیاده سازی شما تفاوت بسیار کمی با آنچه در یک وضعیت دنیای واقعی مورد نیاز است، خواهد داشت. شما کافی است نسخه پروتکل بیت متناوب (Alternating-Bit-Protocol) این آزمایشگاه را با زبان C یا پایتون پیاده سازی کنید. starter code مورد نیاز هر کدام در اختیار شما قرار میگیرد.

از آنجایی که شما احتمالاً ماشین های مستقلی (با سیستم عاملی که بتوانید آن را تغییر دهید) در اختیار ندارید، کد شما باید در یک محیط سخت افزاری/نرم افزاری شبیه سازی شده اجرا شود. با این حال، رابط برنامه نویسی ارائه شده به روتین های شما، یعنی کدی که موجودیت های شما را از بالا و از پایین فراخوانی می کند، بسیار نزدیک به آن چیزی است که در یک محیط واقعی UNIX انجام می شود. (در واقع، رابط های نرم افزاری توصیف شده در این تکلیف برنامه نویسی بسیار واقعی تر از فرستنده ها و گیرنده های حلقه بی نهایتی هستند که بسیاری از متون توصیف می کنند). توقف/شروع تایمرها نیز شبیه سازی شده است، و وقفه های تایمر باعث فعال شدن روتین مدیریت تایمر شما می شوند.

روتین‌هایی که شما خواهید نوشت

پرونده‌هایی که شما خواهید نوشت برای موجودیت فرستنده (A) و موجودیت گیرنده (B) هستند. فقط انتقال یک‌طرفه داده (از A به B) مورد نیاز است. البته، سمت B مجبور خواهد بود برای تایید (مثبت یا منفی) دریافت داده، بسته‌هایی را به A ارسال کند. روتین‌های شما باید به شکل پرونده‌هایی که در زیر توضیح داده شده‌اند، پیاده‌سازی شوند. این پرونده‌ها توسط پرونده‌هایی که ما نوشته‌ام و یک محیط شبکه را شبیه‌سازی می‌کنند، فراخوانی خواهند شد (و آن‌ها را فراخوانی خواهند کرد). ساختار کلی محیط در شکل ۱ (ساختار محیط شبیه‌سازی شده) نشان داده شده است.



شکل ۱: ساختار کلی محیط

واحد داده‌ای که بین لایه‌های بالایی و پروتکل‌های شما پاس داده می‌شود، یک **message** (پیام) است که به این صورت تعریف شده است:

```
struct msg {
char data[20];
};
```

این تعریف، و تمام ساختارهای داده‌ای دیگر و روتین‌های شبیه‌ساز، و همچنین روتین‌های استاب (**stub**) (یعنی آنهایی که شما باید تکمیل کنید) در فایل **prog2.c** قرار دارند که بعداً توضیح داده می‌شود. بنابراین، موجودیت فرستنده شما داده‌ها را در تکه‌های (**chunk**) ۲۰ بایتی از **layer5** دریافت خواهد کرد؛ موجودیت گیرنده شما باید تکه‌های ۲۰ بایتی از داده‌های به درستی دریافت شده را به **layer5** در سمت گیرنده تحویل دهد. واحد داده‌ای که بین روتین‌های شما و لایه شبکه پاس داده می‌شود، **packet** (بسته) است که به این صورت تعریف شده است:

```
struct pkt {
int seqnum;
int acknum;
```

```
int checksum;
char payload[20];
};
```

روتین‌های شما فیلد **payload** را از داده‌های پیامی که از **layer5** پایین آمده است، پر خواهند کرد. فیلدهای دیگر بسته توسط پروتکل‌های شما برای اطمینان از تحویل قابل اعتماد، همانطور که در کلاس دیده‌ایم، استفاده خواهند شد.

روتین‌هایی که شما خواهید نوشت در زیر به تفصیل آمده‌اند. همانطور که در بالا ذکر شد، چنین پروسه‌هایی در زندگی واقعی بخشی از سیستم‌عامل خواهند بود و توسط پروسه‌های دیگر در سیستم‌عامل فراخوانی می‌شوند.

□ **A-output(message)** که در آن **message** ساختاری از نوع **msg** است و حاوی داده‌ای برای ارسال به سمت **B** می‌باشد. این روتین هر زمان که لایه بالایی در سمت فرستنده (**A**) پیامی برای ارسال داشته باشد، فراخوانی خواهد شد. این وظیفه پروتکل شماست که اطمینان حاصل کند داده‌های موجود در چنین پیامی به ترتیب و به درستی به لایه بالایی سمت گیرنده تحویل داده می‌شوند.

□ **A-input(packet)** که در آن **packet** ساختاری از نوع **pkt** است. این روتین هر زمان که بسته‌ای ارسالی از سمت **B** (یعنی در نتیجه اجرای **tolayer3()** توسط یک پروسه در سمت **B**) به سمت **A** برسد، فراخوانی خواهد شد. **packet** بسته (احتمالاً خراب شده) ارسالی از سمت **B** است.

□ **A-timerinterrupt()** این روتین زمانی که تایمر **A** منقضی شود (و در نتیجه یک وقفه تایمر ایجاد کند) فراخوانی خواهد شد. شما احتمالاً می‌خواهید از این روتین برای کنترل ارسال مجدد بسته‌ها استفاده کنید. برای نحوه شروع و توقف تایمر، به **starttimer()** و **stoptimer()** در زیر مراجعه کنید.

□ **A-init()** این روتین یک بار، قبل از اینکه هر یک از روتین‌های دیگر سمت **A** شما فراخوانی شوند، فراخوانی خواهد شد. می‌توان از آن برای انجام هرگونه مقداردهی اولیه مورد نیاز استفاده کرد.

□ **B-input(packet)** که در آن **packet** ساختاری از نوع **pkt** است. این روتین هر زمان که بسته‌ای ارسالی از سمت **A** (یعنی در نتیجه اجرای **tolayer3()** توسط یک پروسه در سمت **A**) به سمت **B** برسد، فراخوانی خواهد شد. **packet** بسته (احتمالاً خراب شده) ارسالی از سمت **A** است.

□ **B-init()** این روتین یک بار، قبل از اینکه هر یک از روتین‌های دیگر سمت **B** شما فراخوانی شوند، فراخوانی خواهد شد. می‌توان از آن برای انجام هرگونه مقداردهی اولیه مورد نیاز استفاده کرد.

رابطه های نرم افزاری

پرونده هایی که در بالا توضیح داده شدند، آنهایی هستند که شما خواهید نوشت. ما روتین های زیر را نوشته ایم که می توانند توسط روتین های شما فراخوانی شوند:

□ `starttimer(calling-entity, increment)` که در آن `calling-entity` یا 0 (برای شروع تایمر سمت A) یا 1 (برای شروع تایمر سمت B) است، و `increment` یک مقدار `float` است که نشان دهنده مقدار زمانی است که قبل از وقفه تایمر سپری خواهد شد. تایمر A فقط باید توسط روتین های سمت A شروع (یا متوقف) شود، و به طور مشابه برای تایمر سمت B. برای اینکه ایده ای از مقدار `increment` مناسب برای استفاده داشته باشید: یک بسته ارسال شده به شبکه به طور متوسط 5 واحد زمانی طول می کشد تا به طرف دیگر برسد، زمانی که هیچ پیام دیگری در رسانه وجود ندارد.

□ `stoptimer(calling-entity)` که در آن `calling-entity` یا 0 (برای توقف تایمر سمت A) یا 1 (برای توقف تایمر سمت B) است.

□ `tolayer3(calling-entity, packet)` که در آن `calling-entity` یا 0 (برای ارسال از سمت A) یا 1 (برای ارسال از سمت B) است، و `packet` ساختاری از نوع `pkt` است. فراخوانی این روتین باعث می شود بسته به شبکه ارسال شود و مقصد آن موجودیت دیگر باشد.

□ `tolayer5(calling-entity, message)` که در آن `calling-entity` یا 0 (برای تحویل به layer 5 در سمت A) یا 1 (برای تحویل به layer 5 در سمت B) است، و `message` ساختاری از نوع `msg` است. با انتقال داده یک طرفه، شما فقط این روتین را با `calling-entity` برابر با 1 (تحویل به سمت B) فراخوانی خواهید کرد. فراخوانی این روتین باعث می شود داده ها به layer 5 پاس داده شوند.

محیط شبکه شبیه سازی شده

فراخوانی پرونده `tolayer3()` بسته ها را به رسانه (یعنی به لایه شبکه) ارسال می کند. پرونده های شما `A-input()` و `B-input()` زمانی فراخوانی می شوند که بسته ای قرار است از رسانه به لایه پروتکل شما تحویل داده شود. رسانه قادر به خراب کردن و گم کردن بسته ها است. ترتیب بسته ها را تغییر نخواهد داد. هنگامی که شما پرونده های خود و پرونده های ما را با هم کامپایل کرده و برنامه حاصل را اجرا می کنید، از شما خواسته می شود تا مقادیری را در مورد محیط شبکه شبیه سازی شده مشخص کنید:

□ تعداد پیام ها برای شبیه سازی. شبیه ساز ما (و روتین های شما) به محض اینکه این تعداد پیام از layer 5 پایین فرستاده شد، متوقف خواهد شد، صرف نظر از اینکه آیا تمام پیام ها به درستی تحویل داده شده اند یا خیر.

بنابراین، نیازی نیست نگران پیام‌های تحویل داده‌نشده یا ACK نشده‌ای باشید که هنگام توقف شبیه‌ساز هنوز در فرستنده شما هستند. توجه داشته باشید که اگر این مقدار را روی 1 تنظیم کنید، برنامه شما بلافاصله، قبل از تحویل پیام به طرف دیگر، خاتمه می‌یابد. بنابراین، این مقدار باید همیشه بزرگتر از 1 باشد.

□ گم شدن (Loss). از شما خواسته می‌شود تا احتمال گم شدن بسته را مشخص کنید. مقدار 0.1 به این معنی است که (به طور متوسط) از هر ده بسته یکی گم می‌شود.

□ خراب شدن (Corruption). از شما خواسته می‌شود تا احتمال خراب شدن بسته را مشخص کنید. مقدار 0.2 به این معنی است که (به طور متوسط) از هر پنج بسته یکی خراب می‌شود. توجه داشته باشید که محتویات فیلدهای ack, sequence, payload یا checksum می‌توانند خراب شوند. بنابراین، checksum شما باید شامل داده‌ها، فیلدهای sequence و ack باشد.

□ ردیابی (Tracing). تنظیم مقدار ردیابی روی 1 یا 2 اطلاعات مفیدی در مورد آنچه در داخل شبیه‌سازی در حال رخ دادن است چاپ می‌کند (مثلاً چه اتفاقی برای بسته‌ها و تایمرها می‌افتد). مقدار ردیابی 0 این قابلیت را خاموش می‌کند. مقدار ردیابی بزرگتر از 2 انواع پیام‌های عجیبی را نمایش می‌دهد که برای اهداف دیباگ کردن خود شبیه‌ساز ما هستند. مقدار ردیابی 2 ممکن است برای شما در دیباگ کردن کدتان مفید باشد. باید به خاطر داشته باشید که پیاده‌سازان واقعی شبکه‌های زیربنایی ندارند که چنین اطلاعات خوبی در مورد آنچه قرار است برای بسته‌هایشان اتفاق بیفتد، ارائه دهند!

□ متوسط زمان بین پیام‌ها از layer5 فرستنده. شما می‌توانید این مقدار را روی هر مقدار مثبت و غیر صفر تنظیم کنید. توجه داشته باشید که هرچه مقدار کمتری انتخاب کنید، بسته‌ها سریع‌تر به فرستنده شما می‌رسند.

نسخه پروتکل بیت متناوب (Alternating-Bit-Protocol) این آزمایشگاه

شما باید پروسه‌های A-output(), A-input(), A-timerinterrupt(), A-init(), B-input() و B-init() را بنویسید که با هم یک انتقال داده یک‌طرفه توقف-و-انتظار (stop-and-wait) (یعنی پروتکل بیت متناوب، که ما در متن کتاب به آن rdt3.0 می‌گوییم) را از سمت A به سمت B پیاده‌سازی کنند. پروتکل شما باید از هر دو پیام ACK و NACK استفاده کند.

شما باید مقدار بسیار بزرگی را برای متوسط زمان بین پیام‌ها از layer5 فرستنده انتخاب کنید، تا فرستنده شما هرگز زمانی که هنوز یک پیام برجسته و تایید نشده دارد که در تلاش برای ارسال آن به گیرنده است، فراخوانی نشود. ما پیشنهاد می‌کنیم مقداری حدود 1000 را انتخاب کنید. همچنین باید در فرستنده خود بررسی کنید تا مطمئن شوید زمانی که A-output() فراخوانی می‌شود، هیچ پیامی در حال حاضر در حال انتقال نیست. اگر وجود دارد، می‌توانید به سادگی داده‌های پاس داده شده به روتین A-output() را نادیده بگیرید (drop کنید).

شما باید پروسه های خود را در فایل به نام prog2.c قرار دهید. شما به نسخه اولیه این فایل، حاوی روتین های شبیه سازی که ما برای شما نوشته ایم، و استاب های (stubs) پروسه های شما، نیاز خواهید داشت. می توانید این برنامه را از صفحه درس دریافت کنید.

این آزمایشگاه را می توان بر روی هر ماشینی که از C پشتیبانی می کند، تکمیل کرد. این برنامه از هیچ ویژگی UNIX استفاده نمی کند. (شما می توانید به سادگی فایل prog2.c را به هر ماشین و سیستم عاملی که انتخاب می کنید کپی کنید).

ما توصیه می کنیم که یک لیست کد، یک سند طراحی، و خروجی نمونه را تحویل دهید. برای خروجی نمونه شما، پروسه های شما می توانند هر زمان که رویدادی در فرستنده یا گیرنده شما رخ می دهد (ورود یک پیام/بسته، یا یک وقفه تایمر) و همچنین هر اقدامی که در پاسخ انجام می شود، پیامی را چاپ کنند. شاید بخواهید خروجی را برای یک اجرا تا نقطه ای (تقریباً) که 10 پیام به درستی در گیرنده ACK شده اند، با احتمال گم شدن 0.1 و احتمال خرابی 0.3 و سطح ردیابی 2 تحویل دهید. شاید بخواهید پرینت خروجی خود را با یک خودکار رنگی حاشیه نویسی کنید که نشان می دهد چگونه پروتکل شما به درستی از گم شدن و خرابی بسته بازیابی شده است. اطمینان حاصل کنید که "نکات مفید" (helpful hints) این آزمایشگاه را می خوانید.

نسخه Go-Back-N این آزمایشگاه (امتیازی)

شما باید رویه های A_output(), A_input(), A_timerinterrupt(), A_init(), B_input() و B_init() را بنویسید که روی هم رفته یک انتقال داده یک طرفه Go-Back-N را از فرستنده A به گیرنده B با اندازه پنجره ۸ پیاده سازی کنند. پروتکل شما باید از پیام های ACK و NACK استفاده کند. برای اطلاعات در مورد نحوه دستیابی به شبیه ساز شبکه، به نسخه پروتکل بیت متناوب (alternating-bit-protocol) این آزمایشگاه در بالا مراجعه کنید. ما اکیداً توصیه می کنیم که ابتدا آزمایشگاه آسان تر (بیت متناوب - Alternating Bit) را پیاده سازی کنید و سپس کد خود را برای پیاده سازی آزمایشگاه سخت تر (Go-Back-N) گسترش دهید. باور کنید - این کار اتلاف وقت نخواهد بود! با این حال، چند ملاحظه جدید برای کد Go-Back-N شما (که در مورد پروتکل بیت متناوب صدق نمی کنند) عبارتند از:

□ A_output(message) که در آن message ساختاری از نوع msg است و حاوی داده هایی برای ارسال

به گیرنده B می باشد. روال A_output() شما اکنون گاهی اوقات زمانی فراخوانی می شود که پیام های در حال انتظار و تأیید نشده ای در رسانه وجود دارد - این بدان معناست که شما باید چندین پیام را در فرستنده خود بافر کنید. همچنین، به دلیل ماهیت Go-Back-N، به بافرینگ در فرستنده خود نیاز خواهید داشت: گاهی اوقات فرستنده شما فراخوانی می شود اما نمی تواند پیام جدید را ارسال کند زیرا پیام جدید خارج از پنجره قرار می گیرد.

□ به جای اینکه نگران بافر کردن تعداد نامحدودی از پیامها باشید، اشکالی ندارد که تعداد محدود و مشخصی بافر در فرستنده خود داشته باشید (مثلاً برای ۵۰ پیام) و اگر هر ۵۰ بافر در یک لحظه در حال استفاده بودند، فرستنده شما به سادگی **abort** کند (تسلیم شود و خارج شود) (توجه: با استفاده از مقادیر داده شده در ادامه، این اتفاق هرگز نباید رخ دهد!) البته در «دنیای واقعی»، باید راه حل ظریفتری برای مشکل بافر محدود پیدا کرد!

□ **A_timerinterrupt()** این روال زمانی فراخوانی می شود که تایمر **A** منقضی شود (و در نتیجه یک وقفه تایمر ایجاد کند). به یاد داشته باشید که شما فقط یک تایمر دارید و ممکن است بسته های در حال انتظار و تأیید نشده زیادی در رسانه داشته باشید، بنابراین باید کمی در مورد نحوه استفاده از این تایمر واحد فکر کنید.

برای توضیحات کلی در مورد آنچه ممکن است بخواهید تحویل دهید، به نسخه پروتکل بیت متناوب (**Alternating-bit-protocol**) این آزمایشگاه در بالا مراجعه کنید. شاید بهتر باشد خروجی اجرایی را تحویل دهید که به اندازه کافی طولانی بوده تا حداقل ۲۰ پیام با موفقیت از فرستنده به گیرنده منتقل شده باشند (یعنی فرستنده برای این پیامها **ACK** دریافت کند)، با احتمال گم شدن ۰.۲، احتمال خرابی ۰.۲، سطح ردیابی (**trace level**) ۲، و میانگین زمان بین ورودها ۱۰. همچنین می توانید بخش هایی از خروجی چاپی خود را با یک خودکار رنگی حاشیه نویسی کنید که نشان دهد چگونه پروتکل شما به درستی از گم شدن و خرابی بسته ها باز یابی شده است.

نکته:

می توانید انتقال دوطرفه پیامها را پیاده سازی کنید. در این حالت، موجودیت های **A** و **B** هم به عنوان فرستنده و هم به عنوان گیرنده عمل می کنند. همچنین می توانید تأییدیه ها را روی بسته های داده **piggyback** کنید (یا می توانید این کار را نکنید). برای اینکه شبیه ساز من پیامها را از لایه ۵ به روال **B_output()** شما تحویل دهد، باید مقدار تعریف شده **BIDIRECTIONAL** را از ۰ به ۱ تغییر دهید.

نکات مفید و موارد مشابه

□ چکسام کردن (**Checksumming**). شما می توانید از هر رویکردی که برای **checksumming** می خواهید استفاده کنید. به یاد داشته باشید که شماره توالی و فیلد **ack** نیز می توانند خراب شوند. ما یک **checksum** شبیه به **TCP** را پیشنهاد می کنیم، که شامل مجموع مقادیر (صحیح) فیلدهای **sequence** و **ack** است، که به مجموع کاراکتر به کاراکتر فیلد **payload** بسته اضافه شده است (یعنی با هر کاراکتر طوری رفتار کنید که گویی یک عدد صحیح ۸ بیتی است و فقط آنها را با هم جمع کنید).

□ توجه داشته باشید که هر «وضعیت» مشترک بین پروسه‌های شما باید به شکل متغیرهای سراسری باشد. همچنین توجه داشته باشید که هر اطلاعاتی که پروسه‌های شما باید از یک فراخوانی به فراخوانی بعدی ذخیره کنند، باید یک متغیر سراسری (یا ایستا) نیز باشد. به عنوان مثال، پروسه‌های شما باید یک کپی از یک بسته را برای ارسال مجدد احتمالی نگه دارند. احتمالاً ایده خوبی است که چنین ساختار داده‌ای یک متغیر سراسری در کد شما باشد. با این حال، توجه داشته باشید که اگر یکی از متغیرهای سراسری شما توسط سمت فرستنده استفاده می‌شود، آن متغیر نباید توسط موجودیت سمت گیرنده قابل دسترسی باشد، زیرا در زندگی واقعی، موجودیت‌های ارتباطی که فقط از طریق یک کانال ارتباطی به هم متصل هستند، نمی‌توانند متغیرهای سراسری را به اشتراک بگذارند.

□ یک متغیر سراسری `float` به نام `time` وجود دارد که می‌توانید از داخل کد خود به آن دسترسی داشته باشید تا در پیام‌های تشخیصی به شما کمک کند.

□ ساده شروع کنید (**START SIMPLE**). احتمالات گم شدن و خرابی را صفر تنظیم کنید و روتین‌های خود را آزمایش کنید. بهتر از آن، پروسه‌های خود را برای حالت بدون گم شدن و بدون خرابی طراحی و پیاده‌سازی کنید و ابتدا آنها را به کار بیندازید. سپس حالت یکی از این احتمالات غیر صفر را مدیریت کنید، و در نهایت هر دو را غیر صفر در نظر بگیرید.

□ دیباگ کردن (**Debugging**). ما توصیه می‌کنیم که سطح ردیابی را روی 2 تنظیم کنید و در حین دیباگ کردن پروسه‌های خود، `printf`‌های زیادی در کد خود قرار دهید.

□ اعداد تصادفی (**Random Numbers**). شبیه‌ساز با استفاده از یک تولیدکننده اعداد تصادفی، گم شدن و خطاهای بسته را ایجاد می‌کند. تجربه گذشته ما این است که تولیدکننده‌های اعداد تصادفی می‌توانند از یک ماشین به ماشین دیگر بسیار متفاوت باشند. ممکن است نیاز باشد کد تولید اعداد تصادفی را در شبیه‌سازی که ما به شما عرضه کرده‌ایم، تغییر دهید. روتین‌های شبیه‌سازی ما یک تست دارند تا ببینند آیا تولیدکننده اعداد تصادفی روی ماشین شما با کد ما کار می‌کند یا خیر. اگر پیغام خطایی دریافت کردید:

```
It is likely that random number generation on your machine is different
from what this emulator expects. Please take a look at the routine
jimsrand() in the emulator code. Sorry.
```

در این صورت خواهید دانست که باید به نحوه تولید اعداد تصادفی در روتین `jimsrand()` نگاهی بیندازید؛ به توضیحات (`comments`) در آن روتین مراجعه کنید.

موارد قابل تحویل

۱. از starter code یکی از فایل های سی یا پایتون را انتخاب کرده و فایل تکمیل شده آن را تحویل دهید.
۲. فقط کافی است یکی از زبان های سی یا پایتون را تحویل دهید. انجام هر دو آنها نمره اضافی ندارد!
۳. برای بخش امتیازی حتما فایل جداگانه با نام Bonus-gbn.py آپلود کنید.

نکاتی که باید توجه داشته باشید:

- الف) مهلت ارسال در سربرگ تمرین همچنین در ایلرن درج شده است.
- ب) کلیه تمرینات (غیر از تمرینات سرکلاسی) از طریق ایلرن دریافت می شوند و دیگر شیوه های ارسال تمرین پذیرفته نیست.
- ج) در صورتی که تمرینات در قالب $\text{\LaTeX}$ و تنها در **Template** تمرینات ارسال شوند، ۵٪ نمره ی اضافی به عنوان امتیاز در نظر گرفته خواهد شد. (**Template** در ایلرن در دسترس است).
- د) فایل تمرینات ارسالی باید شامل فایل های مورد نیاز باشد. در صورتی که تمرینات در قالب $\text{\LaTeX}$ تهیه شده باشند، لازم است فایل های مورد نیاز جهت اجرای فایل $\text{\LaTeX}$ به همراه فایل PDF نیز ارسال شوند. نام فایل را به صورت زیر انتخاب کنید:

CN#_StudentNumber#_StudentName

- ه) ارسال با تأخیر تمرین ها و پروژه های درس صرفاً بر اساس قوانین ذکر شده در فصل صفر امکان پذیر خواهد بود.